Distributed Event Log with Dynamic Consumer Rebalancing

Project #49 — Project Write-Up

Course: Distributed Systems | Team: _____ | Date: October 16, 2026

1. Project Description

This project implements a Kafka-inspired distributed event log: a partitioned, replicated, append-only commit log with consumer groups, persistent offsets, and dynamic partition rebalancing. Producers publish to a topic; a key-hash partitioner maps each record to one of P partitions, each an independently ordered sequence of offsets stored on disk as segment files with a sparse index. Keying by entity identifier keeps all events for one entity ordered while letting unrelated entities proceed in parallel. Each partition is replicated across R brokers in a leader/follower arrangement: writes go to the leader, followers pull and acknowledge, and a record becomes *committed* once a quorum of the in-sync replica set has persisted it. Consumers read only committed records, so anything read is guaranteed to survive the loss of any single broker.

Consumption is coordinated through consumer groups, in which each partition is owned by exactly one consumer at a time. A group coordinator tracks liveness via heartbeats and recomputes the partition–consumer assignment whenever a member joins, leaves gracefully, or fails to heartbeat within its session timeout. Every assignment carries a monotonically increasing **ownership generation**, and offset commits are validated against the current generation. This fences the central failure mode of such systems: a consumer declared dead but in fact merely frozen — a long GC pause or network stall — cannot wake and commit a stale offset that would rewind the partition beneath its new owner. Its late commit is rejected with a stale-epoch error and the committed offset never moves backwards. This mechanism is the correctness centrepiece of the project.

The system targets **at-least-once** delivery under **backpressure**. Consumers fetch into a bounded in-flight queue and advertise remaining capacity as an explicit credit in the fetch protocol; a saturated consumer stops fetching, which throttles the broker and ultimately the producers, so that backlog accumulates in the on-disk log rather than in memory. Because a record is processed before its offset is committed, a consumer crashed in that window reprocesses on recovery; a scripted crash between processing and commit exposes and quantifies this, while recovery always resumes from the last committed offset and never skips a committed event. Evaluation runs on a single host with brokers and consumers isolated as resource-capped containers and inter-node latency emulated, preserving every failure mode of interest while holding all comparisons under identical conditions.

2. Goals

- Build a partitioned, replicated event log with durable, offset-addressable segment storage and per-partition ordering.
- Implement consumer groups with a coordinator that reassigns partitions on join, graceful leave, and heartbeat-timeout failure.
- Enforce correctness through generation-based fencing: accept offset commits only from the current partition owner, rejecting those from expired generations.
- Persist offsets so a restarted consumer resumes from the last committed offset with no committed event skipped, and apply bounded queues with explicit credit-based backpressure to keep memory stable under sustained overload.
- Demonstrate at-least-once semantics by crashing a consumer between processing and commit, and quantify the resulting duplication.
- Measure rebalance time, backlog drain rate, duplication factor and latency under failure, and compare eager against sticky partition assignment.

3. Tools and Technologies

The system is implemented in **C++20** for direct control over file I/O and memory layout — the project is in substance a storage engine — and for latency measurements undistorted by garbage collection; Redpanda, a C++ reimplementation of Kafka, is precedent. Networking uses blocking sockets with one thread per connection, which suffices for the ∼30 connections of a representative cluster and removes the coroutine lifetime hazards that would otherwise dominate debugging. The coordinator is single-threaded behind an inbound queue, serialising every join, heartbeat and commit so generation updates are atomic by construction rather than by locking.

Layer	Technology
Language / build	C++20 (GCC 13+ / Clang 17+), CMake 3.25+ with `FetchContent`
Transport	POSIX sockets, blocking, thread-per-connection; 4-byte big-endian length prefix, CRC32C framing
Encoding	`nlohmann/json` control plane (debuggability); MessagePack record payloads (volume)
Log storage	Hand-built append-only segment files via `pwrite`, sparse offset index, segment rolling, size/time retention
Offset store	SQLite — a transactional compare-and-set on `(partition, generation)` implements fencing race-free in one statement
Coordination	Self-built single-instance group coordinator (SPOF documented in the report)

Layer	Technology
Replication	Leader/follower with in-sync replica set and quorum acknowledgement; leader appointed by coordinator
Assignment	Round-robin, then sticky — both retained so the comparison can be measured
Fault injection	`SIGSTOP` (frozen consumer), `SIGKILL` (crash), `--crash-after-process` flag (deterministic duplicates), `tc netem`/Toxiproxy (network)
Testing	GoogleTest/Catch2, RapidCheck for invariants; ASan + UBSan in development, ThreadSanitizer in CI, `rr` for race replay
Observability	`spdlog` structured logging, CSV metric export, pandas + matplotlib for figures
Workflow	Git/GitHub with CI, `clang-format`/`clang-tidy`, Docker Compose for evaluation

4. Special System Requirements

The project runs end-to-end — implementation, demonstrations and measurement — on a single laptop. Brokers and consumers are ordinary processes with independent data directories and ports communicating over TCP, each killable or freezable independently, which reproduces every failure mode the project studies. No GPUs, licensed software or external datasets are required; all workloads are synthetically generated.

- **Baseline:** 4+ cores (8 comfortable), 8 GB RAM (16 comfortable), 20 GB free *local* SSD. A representative run hosts 3 brokers plus 4–8 consumers concurrently. Local SSD is required because segments are `fsync`ed; a network-mounted home directory would distort durability measurements.
- **Toolchain:** GCC 13+ or Clang 17+ for full C++20 support. Lab machines frequently ship older compilers; where the system toolchain is insufficient we build inside a container with a pinned image.
- **Privileged access:** network fault injection with `tc netem` requires `NET_ADMIN` capability inside containers. Where elevated privilege is unavailable, Toxiproxy provides equivalent userspace fault injection and is used instead.
- **Research Computing Environment (optional):** requested only to raise fidelity of the scaled runs, placing brokers and consumers on separate physical nodes so replication lag reflects real network behaviour rather than loopback. A 3–4 node allocation with 8 cores and 16 GB per node is sufficient. **If RCE access is unavailable these runs degrade to a single host with emulated latency at reduced scale; no deliverable depends on cluster access.**

5. Expected Outcomes

The primary result is a working event log that preserves its correctness invariants under injected failure, with a quantitative characterisation of recovery behaviour. E5 and E8 are the headline demonstrations.

#	Experiment	Demonstrates	Expected result
E1	Steady-state produce/consume	partitioning, replication	stable throughput; per-partition order never violated
E2	Kill one consumer	coordination	rebalance time ≈ session timeout + <500 ms
E3	Sweep session timeout 2–15 s	coordination	near-linear; false evictions below ∼2 s
E4	Eager vs. sticky assignment	coordination	sticky 40–70% faster to full throughput
E5	**Freeze consumer, let it wake**	**fencing**	**0 offset regressions with fencing; >0 without**
E6	Kill partition leader	replication	new leader elected; 0 committed events lost
E7	Producer at 3× consumer capacity	backpressure	in-flight depth and RSS flat at bound; backlog on disk
E8	**Crash between process and commit**	**at-least-once**	**duplicates ≤ commit interval, mean ≈ half**
E9	Backlog drain after E2	coordination	drain rate ≈ consumer capacity − producer rate

Correctness results are binary and asserted in the test suite: zero offset regressions under fencing, zero committed events skipped across crash-and-recovery runs, no partition ever owned by two consumers at once, and zero committed-event loss on leader failover at replication factor 3. **Quantitative results** are reported as plots with raw data: rebalance time against session timeout and group size, the eager/sticky comparison, duplicates as a linear function of commit interval (quantifying the cost of at-least-once), bounded in-flight depth and memory under overload, and drain rate after rebalance. Aggregate throughput is reported as context, not as a result, since on a shared host it is bounded by `fsync` policy and CPU contention rather than by the protocol.

Deliverables: (1) a documented C++20 repository with CMake build, broker, coordinator and client library; (2) nine scripted demo scenarios, one per experiment, each runnable by a single command; (3) an automated unit, integration and property-based test suite run in CI under ThreadSanitizer; (4) raw CSV metric exports with the plotting scripts generating every figure; (5) a final report evaluating measurements against the predictions above; and (6) a live demonstration of the fencing and duplicate-delivery scenarios. Predictions contradicted by measurement are reported as such with analysis; negative results are findings, not failures.